

Relatório Projeto 3 - Estrutura de Dados

Calculadora

Disciplina: TI221 - Estrutura de Dados

Curso: Informática Integrado ao Ensino Médio Diurno 19

Prof. Francisco da Fonseca Rodrigues

Alunos:

Gabriel Bellini Camargo - 25131

Pedro Henrique Sakamoto Mendes - 25151

Introdução

Este projeto tem como objetivo o desenvolvimento das habilidades dos alunos na criação de programas por meio da Programação Visual, do Pensamento Orientado a Objetos, do uso de estruturas de dados baseadas em pilhas e da aplicação de algoritmos de conversão e avaliação de expressões matemáticas. Como tema, foi adotado o desenvolvimento de uma Calculadora que opera com notação pósfixa (Reverse Polish Notation), utilizando Windows Forms em C#.

Desenvolvimento

a. Datas, horas e tempo de desenvolvimento

O projeto teve seu desenvolvimento iniciado no dia 22/06 e finalizado no dia 24/06. Durante os dias em que o projeto foi desenvolvido, a duração média do trabalho diário foi de duas horas. Somando o tempo gasto em cada dia para codificar o projeto, chegamos a um total aproximado de 7 horas de desenvolvimento.

b. Erros encontrados e soluções aplicadas

1 - Índice fora dos limites no método `ValorDaExpressaoPosfixa()`.

O primeiro erro ocorreu no método `ValorDaExpressaoPosfixa()`, que recebia a expressão pósfixa em letras e buscava o valor correspondente de cada letra no vetor `ValorDe[ ]` pela expressão símbolo - 'A'. O erro se manifestava como um `IndexOutOfRangeException` porque a expressão pósfixa estava recebendo os números reais diretamente, sem ter passado pela etapa de substituição por letras feita pelo método `PrepararExpressao()`. Isso ocorria porque o método `btnIgual_Click` estava chamando `ConverterInfixaParaPosfixa()` diretamente sobre `txtVisor.Text`, pulando a etapa de `PrepararExpressao()`.

A solução foi garantir que o fluxo de execução no botão de igual seguisse a ordem correta: primeiro tokenizar e substituir por letras e depois converter para pósfixa.

2 - Parênteses desbalanceados não validados

O segundo erro ocorria quando o usuário digitava expressões com parênteses desbalanceados, como `10.9+8)` ou `(5+3`. O método `ConverterInfixaParaPosfixa()` não verificava se ao encontrar um ')' existia um

'(' correspondente na pilha, nem verificava se ao final da conversão restavam '(' ainda empilhados. Como resultado, o programa tentava desempilhar de uma pilha vazia, lançando uma exceção que demonstrava um underflow da pilha sem nenhuma mensagem clara ao usuário. A solução foi adicionar duas validações: ao encontrar ')', verifica se a pilha está vazia antes de desempilhar; e ao final, verifica se algum '(' permanece na pilha. Em ambos os casos, uma exceção descritiva é lançada com a mensagem "Parênteses desbalanceados!".

3 - Expressão malformada não tratada ao avaliar pósfixa

O terceiro erro ocorria quando o usuário digitava expressões incompletas, como $8+$ (operador sem operando direito). O método `ValorDaExpressaoPosfixa()` tentava desempilhar dois operandos para cada operador sem verificar se a pilha possuía elementos suficientes. Isso resultava em um underflow da pilha sem contexto. A solução foi adicionar verificações antes de cada `Desempilhar()` no caso dos operadores, lançando uma exceção "Expressão inválida!" se a pilha não tivesse operandos o suficiente.

c. Dificuldades encontradas

Uma das principais dificuldades encontradas foi compreender e implementar corretamente o fluxo completo de três etapas exigido pelo algoritmo: tokenização da expressão real, conversão infixa-para-pósfixa e avaliação da pósfixa. A separação dessas responsabilidades em métodos distintos exigiu atenção redobrada para garantir que cada etapa recebesse os dados no formato esperado.

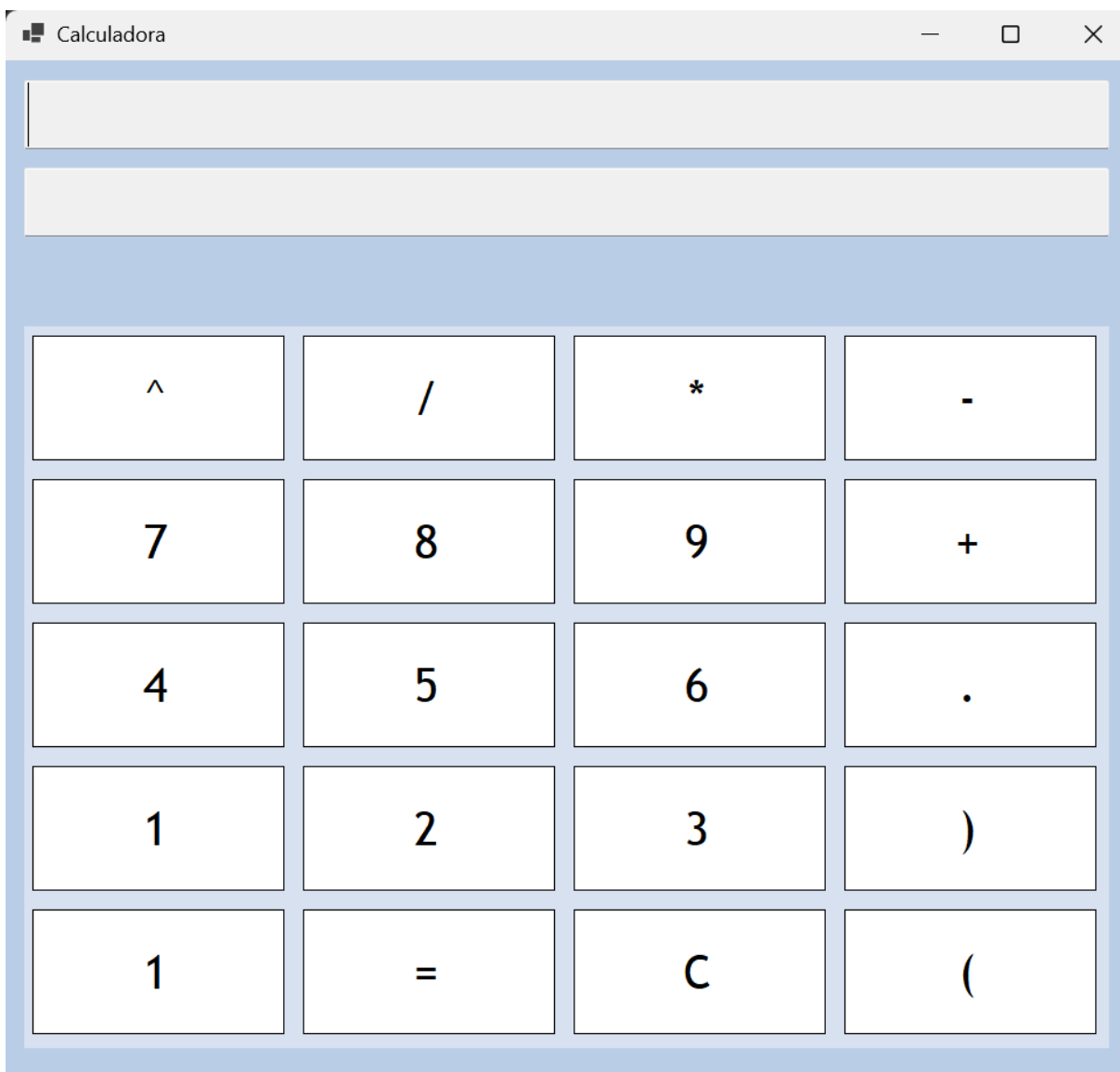
Outra dificuldade foi lidar com expressões que continham números de múltiplos dígitos e decimais, como 99 ou 9.99, que não podiam ser mapeados para um único caractere diretamente. A solução de associar todos os números a letras antes do processamento resolveu essa limitação, mas exigiu a implementação cuidadosa do laço interno em `PrepararExpressao()`.

Por fim, a depuração dos erros foi dificultada pelo fato de as exceções ocorrerem em métodos intermediários, tornando difícil rastrear qual etapa do pipeline estava recebendo dados inválidos. A adição de validações e mensagens de exceção mais descritivas foi essencial para localizar e resolver os problemas com eficiência.

d. Uso de Inteligência Artificial

Não houve uso de inteligência artificial, uma vez que o professor demandou que essa ferramenta não fosse utilizada, sendo que quaisquer dúvidas que surgiram foram sanadas através de pesquisas próprias.

e. Resultado final do layout



Conclusão

Em suma, durante o desenvolvimento desta calculadora com notação pósfixa, trabalhamos na codificação de um projeto que integra estruturas de dados (pilha), algoritmos clássicos de compiladores (conversão infixa-pósfixa e avaliação pósfixa) e programação visual com Windows Forms. Diante desse cenário, a

principal habilidade aprimorada foi a capacidade de depurar uma aplicação de múltiplas etapas interdependentes, identificando em qual fase os dados estavam sendo corrompidos ou mal formatados.

Além disso, o projeto reforçou a importância de separar responsabilidades em métodos bem definidos e de garantir que cada método receba seus dados no formato correto, evitando que erros de uma etapa se propaguem silenciosamente para as seguintes. Por fim, a experiência de reutilizar métodos existentes — como `EhOperador()` dentro dos métodos — demonstrou na prática o valor de escrever códigos coesos e reutilizáveis.